

MySQL 与 PostgreSQL

高级面试题完整答案

从事务、MVCC、索引与优化器到复制、高可用和数据库架构

高级 Java / 数据库开发面试复习讲义

2026-06

阅读说明

本讲义面向具有 Java 后端、微服务、数据平台或数据库运维经验的中高级工程师。内容按照真实面试中的表达顺序组织：**先给结论，再解释底层机制，然后给出 SQL、执行计划或排障命令，最后讨论生产边界和方案取舍。**

版本语境

- MySQL 以 **MySQL 8.4 LTS + InnoDB** 为主要语境，同时覆盖大量生产环境仍在使用的 8.0。MySQL Innovation 版本迭代更快，但面试回答不应把某个 Innovation 版本的新特性当成所有 8.x 环境都具备。
- PostgreSQL 以 **PostgreSQL 18** 为主要语境，并尽量使用 PostgreSQL 14-18 均成立的通用机制。PostgreSQL 19 在 2026 年 6 月仍处于 Beta 阶段，不作为生产默认语境。
- 文中“PG”是 PostgreSQL 的简称；“MySQL”如无特别说明均指 InnoDB，而不是 MyISAM、NDB 等其他存储引擎。
- 参数默认值会随版本、发行版和云厂商变化。面试中应解释参数的作用与权衡，不应只背一个固定数字。

推荐的面试回答结构

面对原理题，建议按以下五步回答：

1. **一句话结论**：先明确两者相同点和不同点。
2. **关键数据结构**：页、索引、版本链、日志或锁如何组织。
3. **执行链路**：一次读写、提交或复制如何流转。
4. **边界与反例**：什么情况下结论不成立，可能出现什么副作用。
5. **生产实践**：如何设计、监控、排障和验证。

面对故障题，建议使用“现象确认 → 缩小范围 → 找等待或资源瓶颈 → 证据闭环 → 止损 → 根因修复 → 防复发”的顺序，而不是一上来就修改参数。

目录

1. MySQL 与 PostgreSQL 的核心架构有什么差异，业务中如何选型
2. 一次事务提交经历了什么，redo/binlog 与 WAL 有什么区别
3. MySQL 与 PostgreSQL 的 MVCC 分别如何实现
4. 隔离级别、幻读和写偏差应如何理解
5. 行锁、间隙锁、谓词锁和死锁有什么差异
6. 数据类型、主键、时间、字符集与 NULL 应如何设计
7. 两者的 B-Tree 索引有什么本质差异
8. 联合索引、覆盖索引、部分索引与表达式索引如何设计
9. JOIN、子查询、CTE 和窗口函数如何选择
10. 分区表什么时候有价值，为什么不能把分区当万能优化
11. 如何阅读 MySQL EXPLAIN ANALYZE 与 PostgreSQL EXPLAIN
12. 优化器统计信息为什么会导导致错误执行计划
13. 一条慢 SQL 应如何系统优化
14. 深分页、COUNT、ORDER BY 和 Top-N 应如何优化
15. 大批量写入、在线 DDL 和索引构建如何控制风险
16. InnoDB Buffer Pool、redo、undo、doublewrite 与 checkpoint 如何协作
17. PostgreSQL VACUUM、autovacuum、HOT 与表膨胀如何理解
18. MySQL 线程模型、PG 进程模型与连接池如何设计
19. MySQL 主从复制、GTID、半同步和 Group Replication 如何理解
20. PostgreSQL 流复制、同步复制、逻辑复制和 replication slot 如何理解
21. 备份、恢复与 PITR 应如何设计并验证
22. 高可用切换如何避免数据丢失、脑裂和读到旧数据
23. 数据库 CPU、IO、锁等待或延迟突然升高时如何排查
24. MySQL 与 PostgreSQL 如何做不停机迁移
25. 场景设计：构建高并发订单与账户数据库
26. 附录：对比速查、排障 SQL、容量检查清单与官方资料

第一部分：共同基础与核心差异

1. MySQL 与 PostgreSQL 的核心架构有什么差异，业务中如何选型？

1.1 结论先行

MySQL 与 PostgreSQL 都是成熟的关系型数据库，但设计重心不同：

- **MySQL/InnoDB** 在互联网 OLTP、读写分离、生态成熟度、运维标准化和开发者普及度方面优势明显。
- **PostgreSQL** 在 SQL 能力、数据类型、扩展性、复杂查询、GIS、部分索引、表达式索引和标准兼容方面更强。

选型不能简化成“谁性能更高”。相同硬件上的性能取决于数据模型、SQL、并发模式、持久化设置和运维能力。更合理的问题是：**哪一种数据库更贴合业务约束，团队能否稳定运维，迁移和扩展成本如何。**

1.2 进程与组件架构

MySQL 服务端通常是单个 `mysqld` 进程，内部使用线程处理连接和后台任务。SQL 层负责连接、解析、优化、权限等，存储引擎层由 InnoDB 完成页管理、索引、事务、锁和恢复。

```
Client
-> MySQL Server SQL Layer
    -> Parser / Optimizer / Executor
    -> InnoDB Storage Engine
        Buffer Pool / Redo / Undo / Locks / B+Tree
```

PostgreSQL 传统架构是一个 `postmaster` 主进程加多个子进程。通常每个客户端连接对应一个 `backend` 进程，另有 `checkpointer`、`background writer`、`WAL writer`、`autovacuum`、`WAL sender` 等后台进程。

```
Client
-> Backend Process
    -> Parser / Rewriter / Planner / Executor
    -> Shared Buffers / Heap / Index
    -> WAL
Background: Checkpointer / Autovacuum / WAL Writer / Archiver ...
```

这直接影响连接治理：PG 创建大量连接的内存和进程调度成本通常更明显，生产中常使用 `PgBouncer`；MySQL 同样不能无限增加连接，也需要连接池和并发控制。

1.3 存储组织差异

InnoDB 表是**索引组织表**：聚簇索引的叶子节点保存整行记录。若有主键，主键通常是聚簇索引；二级索引叶子保存二级键和主键值。

PostgreSQL 默认是**堆表 + 独立索引**：表中的行版本存储在 heap page，索引条目指向物理元组位置 TID=(block, offset)。更新一行时通常生成新的行版本，因此可能出现死元组，需要 VACUUM 清理。

这意味着：

- InnoDB 按主键范围扫描通常天然连续，二级索引回表需要再按主键查聚簇索引。
- PG 索引与 heap 分离，Index Scan 可能随机访问 heap；Index Only Scan 能否真正不访问 heap，还取决于 visibility map。
- InnoDB 主键宽度会复制到所有二级索引中；PG 的二级索引不复制业务主键，但会维护 TID，并受行版本变化影响。

1.4 功能与生态对比

维度	MySQL / InnoDB	PostgreSQL
默认事务隔离	REPEATABLE READ	READ COMMITTED
表组织	聚簇索引组织	Heap + 独立索引
MVCC 旧版本	Undo 版本链	表中保留多个 tuple 版本
SQL 能力	主流 OLTP 足够，生态普及	更接近 SQL 标准，复杂 SQL 强
索引类型	B-Tree、全文、空间、多值等	B-Tree、Hash、GIN、GiST、SP-GiST、BRIN 等
特殊索引	函数索引、不可见索引	表达式、部分、覆盖、并发构建等
JSON	原生 JSON + 生成列/多值索引	JSON/JSONB + GIN、表达式索引
扩展能力	插件与组件体系	Extension 生态非常强，例如 PostGIS
复制	Binlog 异步/半同步、Group Replication	物理流复制、同步复制、逻辑复制
常见应用	互联网交易、通用 OLTP	复杂业务、数据平台、GIS、多模型需求

1.5 典型选型建议

更倾向 MySQL 的情况：

- 团队已有成熟的 MySQL 运维、监控、分库分表和中间件体系。
- 业务是典型高并发点查、短事务、简单 Join 的 OLTP。
- 上下游、云服务、审计工具和人才储备都以 MySQL 为主。

更倾向 PostgreSQL 的情况：

- 复杂 SQL、窗口分析、递归查询、丰富约束和高级数据类型很多。
- 需要 PostGIS、全文检索、数组、range、JSONB、扩展或自定义类型。
- 希望利用部分索引、表达式索引、事务性 DDL 等能力减少应用复杂度。

不应仅凭以下理由选型：

- “MySQL 一定比 PG 快” 或 “PG 一定比 MySQL 强”。
- “某个大厂在用，所以我们也该用”。
- 只做单条 SQL 的微基准测试，不考虑并发、恢复、备份和团队能力。

1.6 面试追问

问：已经使用 MySQL，是否应该为了高级功能迁移 PG？

先计算迁移收益：功能是否真的能减少长期复杂度，驱动、SQL、数据类型、运维、备份、监控和人才成本是否可控。若只是个别分析查询，更可能通过数仓、只读副本或专用服务解决，而不是全量替换核心主库。

问：PG 更符合 SQL 标准，是否意味着迁移 SQL 很容易？

不意味着。自增、布尔、时间、字符比较、大小写、隐式转换、ON DUPLICATE KEY、JSON 函数、存储过程、锁语义和 DDL 都可能不兼容。

2. 一次事务提交经历了什么，redo/binlog 与 WAL 有什么区别？

2.1 结论先行

两者都使用 Write-Ahead Logging：在脏数据页写回数据文件之前，先把描述修改的日志持久化。这样崩溃后可以通过日志重放恢复已提交修改。

但日志角色不同：

- InnoDB redo log 是存储引擎层的物理/生理日志，用于崩溃恢复；MySQL binlog 是 Server 层的逻辑复制日志，用于复制、CDC 和时间点恢复。
- PostgreSQL WAL 同时承担崩溃恢复、物理复制、归档和 PITR 的基础角色；逻辑复制再通过 logical decoding 从 WAL 提取逻辑变化。

2.2 MySQL 提交链路

一次更新的简化链路：

```
BEGIN
-> 在 Buffer Pool 中修改数据页
-> 生成 undo，用于回滚和 MVCC
-> 生成 redo，先进入 redo log buffer
-> 生成 binlog cache
COMMIT
-> redo prepare
-> 写入并按策略 fsync binlog
-> redo commit
-> 返回成功
```

因为 redo 和 binlog 属于两个日志系统，MySQL 使用内部两阶段提交协调二者，目标是避免：

- redo 已提交但 binlog 缺失：主库有数据，副本和 CDC 没有。
- binlog 已持久化但 redo 未提交：复制出去了但主库恢复后没有该事务。

`innodb_flush_log_at_trx_commit` 与 `sync_binlog` 影响每次提交是否刷盘。最强持久性通常需要二者均采用每事务 `fsync` 语义，但会增加延迟；降低刷盘频率可提高吞吐，却可能在操作系统或机器故障时丢失最近一段事务。

2.3 PostgreSQL 提交链路

简化流程：

```
BEGIN
-> 修改 shared buffers 中的数据页
-> 产生 WAL record, 写入 WAL buffers
COMMIT
-> 写入 commit record
-> 根据 synchronous_commit 等设置等待 WAL flush
-> 返回成功
后台: checkpoint / background writer 逐步写脏页
```

WAL 记录先于对应数据页落盘。崩溃后从最近 checkpoint 开始重放 WAL，使数据库达到一致状态。

`fsync=off` 或不当关闭 `full_page_writes` 可能破坏崩溃安全，不应作为普通性能调优手段。

`synchronous_commit=off` 与完全关闭 `fsync` 不同：前者主要允许客户端在 WAL 真正刷盘前提前得到成功，数据库结构仍保持一致，但机器崩溃可能丢失近期已确认事务。

2.4 为什么需要 checkpoint

日志不能无限增长，恢复也不能从数据库创建时重放。checkpoint 建立恢复起点，并推动脏页写回。

checkpoint 太频繁：

- 写放大和 IO 峰值增加。
- PostgreSQL 可能产生更多 full-page image。
- MySQL redo 可用空间压力上升时也会迫使更积极刷脏。

checkpoint 太少：

- 日志空间与恢复时间增加。
- 异常重启后的恢复可能更久。

调优目标不是追求“越少越好”，而是控制日志空间、写入平滑度和 RTO。

2.5 日志对比

日志	主要用途	内容特征	典型消费者
InnoDB redo	崩溃恢复	页级物理/生理变化	InnoDB recovery
InnoDB undo	回滚、MVCC	行的旧值/反向操作信息	事务回滚、consistent read、purge
MySQL binlog	复制、CDC、PITR	statement/row/mixed；生产常用 row	Replica、Debezium、mysqlbinlog
PostgreSQL WAL	恢复、物理复制、归档	物理变化记录	Recovery、standby、archive

日志	主要用途	内容特征	典型消费者
PG logical changes	逻辑复制/CDC	从 WAL 解码出的表级变化	subscriber、CDC connector

2.6 常见误区

误区：事务返回成功，数据页一定已经写入表文件。

不需要。只要关键日志已按持久化策略落盘，脏页可以延迟写回，崩溃后通过日志恢复。

误区：binlog 能替代 redo。

不能。binlog 不负责 InnoDB 页级崩溃恢复，也不包含所有内部恢复所需信息。

追问：为什么数据库不直接每次提交都刷所有脏页？

随机写与小 IO 成本过高；WAL/redo 把随机页修改转化为更顺序的日志写，并通过批量刷页提高效率。

3. MySQL 与 PostgreSQL 的 MVCC 分别如何实现?

3.1 结论先行

MVCC 让读操作在很多情况下不必与写操作互相阻塞，但两者保存旧版本的位置不同：

- InnoDB 当前行主要保存在聚簇索引中，通过隐藏事务信息和 undo 版本链重建旧版本。
- PostgreSQL 把多个 tuple 版本直接保存在 heap 中，通过 xmin/xmax、事务快照和可见性规则判断哪个版本可见，VACUUM 负责回收不再需要的死元组。

MVCC 不等于“没有锁”。更新同一行、唯一性检查、外键、DDL 和显式锁仍会等待。

3.2 InnoDB 的版本链

InnoDB 记录包含用于 MVCC 的隐藏信息，可抽象为：

```
current row
-> DB_TRX_ID: 最近修改该行的事务
-> DB_ROLL_PTR: 指向 undo 中的旧版本
    -> older version
        -> even older version
```

一致性读创建 Read View，判断某个版本的事务 ID 对当前快照是否可见；若不可见，就沿 undo 链寻找更早版本。

长事务危害：

- 老快照仍可能需要旧版本，purge 无法回收 undo。
- history list 增长，读旧版本成本上升。
- undo tablespace 变大，备份和恢复压力增加。

因此只读事务也应及时提交，不要让连接长时间处于 **idle in transaction** 的等价状态。

3.3 PostgreSQL 的 tuple 版本

PG 更新通常不是原地覆盖：旧 tuple 被标记不再对新事务可见，同时在 heap 中写入新 tuple。每个版本具有事务可见性相关字段。

```
Heap Page
tuple v1 (xmin=T1, xmax=T2)
tuple v2 (xmin=T2, xmax=...)
```

快照根据正在运行和已提交的事务集合判断版本可见性。旧 tuple 在没有任何事务可能看到后成为 dead tuple，由 VACUUM 清理并使空间可复用。

如果更新没有改变任何索引列，并且页内有空间，PG 可能使用 HOT (Heap-Only Tuple) 更新，避免为新版本创建普通索引条目，通过 tuple 链找到最新可见版本。这能显著降低索引膨胀和写放大。

3.4 快照创建时机差异

MySQL InnoDB 默认 REPEATABLE READ 下，普通一致性读通常在事务第一次 consistent read 时建立快照，后续普通 SELECT 使用相同快照。锁定读和写操作读取的是当前可锁定版本，不能简单理解成所有语句都停留在旧快照。

PostgreSQL 默认 READ COMMITTED 下，每条语句开始时取得新快照，因此同一事务内两次 SELECT 可能看到不同的已提交结果；REPEATABLE READ 下事务通常使用一个稳定快照。

3.5 可见性与空间回收对比

维度	InnoDB	PostgreSQL
当前版本	聚簇索引记录	Heap tuple
旧版本	Undo	Heap 中的旧 tuple
回收者	Purge	VACUUM / autovacuum
长事务影响	阻碍 undo purge	阻碍 dead tuple 清理、冻结推进
更新索引列	维护相关二级索引	新 tuple + 新索引条目
优化机制	change buffer 等	HOT、visibility map 等

3.6 面试易错点

误区：PostgreSQL VACUUM 会把文件自动缩小。

普通 VACUUM 通常把空间标记为可复用，并不把文件尾部空间全部返还给操作系统。VACUUM FULL 会重写表并需要强锁，生产中要谨慎；也可使用在线重写工具或重建方案。

误区：MySQL 更新就是完全原地更新。

过于简单。InnoDB 会维护 redo、undo 和索引，记录格式及页分裂也会带来额外写入。

4. 隔离级别、幻读和写偏差应如何理解？

4.1 结论先行

隔离级别不是“开得越高越好”，而是在并发、异常容忍度和重试成本之间权衡。面试不能只背“脏读、不可重复读、幻读”，还应讨论：

- 一致性读与锁定读是否相同。
- 快照隔离可能出现写偏差。
- 真正可串行化通常需要阻塞或事务回滚重试。

MySQL InnoDB 默认 REPEATABLE READ；PostgreSQL 默认 READ COMMITTED。

4.2 经典异常

异常	描述
脏读	读到其他事务尚未提交的数据
不可重复读	同一事务重复读取同一行，结果改变
幻读	同一谓词范围重复查询，行集合改变
丢失更新	后写覆盖前写，前一个修改被无声丢失
写偏差	两个事务基于同一快照修改不同记录，联合约束被破坏

写偏差示例：值班表要求至少一名医生在线。两个医生分别读取“当前有两人在线”，各自把自己的状态改为离线，修改的是不同行，因此普通行锁冲突可能没有发生，最终无人在线。

4.3 MySQL 隔离语义重点

- READ COMMITTED：每次一致性读通常创建新快照，锁范围往往比 RR 更小，但仍需结合具体 SQL 和唯一性/外键检查理解。
- REPEATABLE READ：同一事务普通一致性读使用稳定快照；锁定范围查询可使用 next-key lock 抑制范围内插入。
- SERIALIZABLE：普通 SELECT 在特定条件下转为共享锁读，并发显著下降。

一个常见误解是：“RR 已经完全解决所有业务幻读和并发问题”。实际上：

- 快照读和当前读语义不同。
- 应用先查后写若没有锁或约束，仍可能发生竞态。
- 多行不变量仍需要唯一约束、锁、原子 SQL 或串行化事务。

4.4 PostgreSQL 隔离语义重点

- READ COMMITTED: 每条语句看到语句开始前已提交的数据; 同一事务后续语句可看到新提交。
- REPEATABLE READ: PG 使用 Snapshot Isolation, 可防止更多现象, 但仍可能有写偏差。
- SERIALIZABLE: 使用 Serializable Snapshot Isolation (SSI), 跟踪读写依赖, 检测可能形成的序列化异常, 并让部分事务以 serialization failure 回滚。

因此 PG SERIALIZABLE 的正确使用方式必须包含重试:

```
BEGIN
-> execute business transaction
COMMIT
if SQLSTATE = 40001:
    rollback, backoff, retry entire transaction
```

4.5 如何保护业务不变量

优先级通常是:

1. 数据库唯一约束、外键、CHECK、排除约束等声明式约束。
2. 单条原子 DML, 例如带条件 UPDATE。
3. 明确的 SELECT ... FOR UPDATE, 锁住真正决定结果的行。
4. SERIALIZABLE + 完整事务重试。
5. 分布式场景再使用幂等、状态机和补偿, 而不是把所有问题都推给数据库锁。

库存扣减推荐:

```
UPDATE sku_stock
SET available = available - 1
WHERE sku_id = ? AND available > 0;
```

根据受影响行数判断成功, 而不是先 SELECT 再 UPDATE。

4.6 追问: 幻读到底是否被 MySQL RR 解决?

严谨回答:

- 对普通一致性读, 同一快照下重复查询通常看不到后来提交的新行, 因此结果可重复。
- 对需要修改或锁定的当前读, InnoDB 通过 next-key locking 在索引范围上防止影响语义的插入。
- 但“业务没有并发异常”不能由一句 RR 保证, 仍需看索引、SQL 类型、锁范围和业务约束。

5. 行锁、间隙锁、谓词锁和死锁有什么差异？

5.1 结论先行

锁通常加在数据库实际访问的数据结构上，而不是开发者脑中想象的“WHERE 条件”。缺少索引可能扩大扫描和锁定范围。

- InnoDB 有 record lock、gap lock、next-key lock、insert intention lock 和意向锁等。
- PostgreSQL 有表级锁、row-level lock，以及用于 SSI 的 predicate locking；PG 的普通 UPDATE 不会使用 MySQL 风格的 gap lock 来实现 REPEATABLE READ。
- 死锁是并发数据库的正常可恢复事件，应用必须捕获并重试；真正需要修复的是高频死锁和不合理的访问顺序。

5.2 InnoDB 锁

Record lock：锁定索引记录。

Gap lock：锁定索引记录之间的间隙，主要阻止插入，不等于锁住某一行。

Next-key lock：record lock + 前方 gap，常用于范围扫描。

Insert intention lock：多个事务准备向同一 gap 的不同位置插入时可以并存，减少无意义串行。

Intention lock：表级标记，表示事务准备在表中持有行级 S/X 锁，帮助快速判断表锁兼容性。

示例：

```
SELECT * FROM orders
WHERE user_id = 100 AND created_at >= '2026-01-01'
FOR UPDATE;
```

若没有合适的 (user_id, created_at) 索引，InnoDB 可能扫描并锁定远超预期的索引记录。锁优化首先是访问路径优化，不是简单缩短 lock wait timeout。

5.3 PostgreSQL 行锁与表锁

PG 的 SELECT ... FOR UPDATE、FOR NO KEY UPDATE、FOR SHARE、FOR KEY SHARE 强度不同。外键操作常涉及 key-share 类锁；理解较细的锁模式可以减少不必要冲突。

DDL 往往需要更强表锁。例如很多 ALTER TABLE 操作会拿 ACCESS EXCLUSIVE，阻塞读写；CREATE INDEX CONCURRENTLY 降低阻塞，但耗时更长、资源开销更高，失败后可能留下 invalid index，需要清理。

PG SERIALIZABLE 的 predicate lock 主要用于检测读写依赖，并不等同于把一个范围像传统锁一样阻塞住；冲突可能以序列化失败结束。

5.4 死锁示例

```
事务 A: 锁 order 1 -> 等待 account 9
事务 B: 锁 account 9 -> 等待 order 1
```

数据库检测到等待环后会回滚一个事务。

治理方法：

- 所有代码按相同顺序访问表和行，例如按主键升序。
- 事务保持短小，不在持锁期间调用远程接口。
- 使用准确索引缩小扫描和锁范围。
- 批处理拆成小批次，减少一次持锁数量。
- 应用对 deadlock/serialization failure 做有上限、带抖动的重试。
- 保留死锁现场，而不是只看应用报错。

MySQL 常用证据：

```
SHOW ENGINE INNODB STATUS\G
SELECT * FROM performance_schema.data_lock_waits;
SELECT * FROM performance_schema.data_locks;
```

PostgreSQL 常用证据：

```
SELECT pid, username, state, wait_event_type, wait_event,
       pg_blocking_pids(pid), query
FROM pg_stat_activity
WHERE datname = current_database();
```

5.5 SKIP LOCKED 的适用边界

MySQL 和 PG 都支持类似 FOR UPDATE SKIP LOCKED 的能力，可实现多消费者任务领取：

```
SELECT id
FROM job_queue
WHERE status = 'READY'
ORDER BY id
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

它适合队列式工作分配，不适合需要完整一致结果的普通业务查询，因为它会跳过被锁行，返回的是不完整视图。还要设计任务租约、超时回收、幂等与失败重试。

第二部分：数据模型、索引与 SQL

6. 数据类型、主键、时间、字符集与 NULL 应如何设计？

6.1 结论先行

数据类型设计直接影响正确性、索引大小、比较语义、网络开销和未来迁移。原则是：使用能表达业务语义的最窄且稳定的类型，而不是为了“省空间”牺牲边界，也不要把所有数据都存成字符串或 JSON。

6.2 主键设计

InnoDB 主键是聚簇索引键，并被复制到每个二级索引叶子中，因此主键应尽量：

- 短、稳定、非空。
- 尽量单调，减少随机页分裂和缓存抖动。
- 不频繁更新。

自增 BIGINT 对单库 OLTP 友好，但分布式生成需考虑全局唯一、趋势递增、信息泄露和分片路由。UUID v4 完全随机，对 InnoDB 聚簇索引不友好；若需要 UUID，可使用有序 UUID/UUIDv7、二进制存储或业务上可排序的 128 位 ID，但仍需基准测试。

PG heap 不按主键物理组织，随机 UUID 对 heap 插入位置的影响不同，但 B-Tree 主键索引仍会面临随机插入、页面分裂和缓存局部性问题。BIGSERIAL 是历史常见写法，现代 PG 更推荐 SQL 标准的 identity column：

```
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

6.3 金额与浮点数

金额不要使用 FLOAT/DOUBLE 承担精确计算。可选择：

- DECIMAL/NUMERIC(p,s)：表达清晰，适合金融精度。
- 最小货币单位的整数：例如分，计算快，但要管理币种与小数位。

PG numeric 是任意精度十进制，精确但计算成本高于整数；MySQL DECIMAL 同样是定点类型。无论哪种，都应明确舍入规则，不应在数据库、Java BigDecimal 和前端之间各自默认舍入。

6.4 时间类型

MySQL：

- DATETIME 表示日历日期时间，不自动按会话时区转换。
- TIMESTAMP 存储和读取会涉及时区转换，范围与版本能力需要确认。

PostgreSQL:

- **timestamp without time zone** 不包含时区语义。
- **timestamp with time zone (timestamptz)** 内部表示绝对时间点，展示时会按会话时区转换；它并不保存原始时区名称。

通用建议：

- 业务事件发生时间优先存绝对时间点，应用层使用 UTC，并明确展示时区。
- 纯日历概念如生日、账期日、门店营业时间不要强行转换成 UTC 时间点。
- 不使用字符串存时间，不依赖服务器本地时区。

6.5 字符集、排序规则和大小写

MySQL 应明确使用 **utf8mb4**，不要把历史上的 **utf8** 当完整 Unicode。collation 决定大小写、重音和排序比较语义，可能影响唯一索引。例如大小写不敏感 collation 下，**ABC** 与 **abc** 可能被视为相等。

PG 数据库编码通常使用 UTF8。排序与比较受 collation、操作符类和 ICU/libc 配置影响。升级操作系统或 ICU 后，排序规则版本变化可能要求重建相关索引。

大小写不敏感查询可采用：

- 规范化列，例如统一 lower-case，并建立表达式/函数索引。
- PG **citext** 扩展，但仍需理解 locale 语义。
- MySQL 合适的 collation 或生成列。

6.6 NULL 语义

NULL 表示未知或不适用，不等于空字符串或 0。SQL 使用三值逻辑：

```
col = NULL    -- 错误写法，结果不是 TRUE
col IS NULL   -- 正确
```

唯一约束对 NULL 的处理有数据库和版本语义差异。PG 支持 **UNIQUE NULLS NOT DISTINCT** 来把多个 NULL 视为冲突；MySQL 常规唯一索引通常允许多个 NULL。面试时不要用模糊记忆回答，应说明需要查看具体约束定义和版本。

6.7 JSON 何时使用

JSON 适合变化快、稀疏、弱约束的扩展属性，不适合替代核心关系模型。核心过滤、关联、唯一性和统计字段应优先结构化。

- MySQL JSON 可配合生成列、函数索引和多值索引。

- PG **jsonb** 可使用 GIN、表达式索引，并有丰富操作符。

风险：字段名失控、类型不一致、无法建立外键、统计信息不足、更新整块对象、查询难以治理。

7. 两者的 B-Tree 索引有什么本质差异？

7.1 结论先行

两者都广泛使用平衡树索引，但“叶子节点存什么”和“如何访问表行”不同：

- InnoDB 聚簇索引叶子就是整行，二级索引叶子包含二级键和主键。
- PG B-Tree 索引叶子保存索引键和 heap TID；表行在 heap 中，更新可能产生新 TID。

因此不能把 MySQL 的“回表”和 PG 的“heap fetch”简单视为完全相同。

7.2 为什么数据库常用 B-Tree

B-Tree/B+Tree 适合磁盘页：

- 高扇出使树高很低。
- 有序，支持等值、范围、排序、最值和前缀条件。
- 页级分裂与合并可维护动态数据。

哈希索引只擅长等值，无法自然支持范围和 ORDER BY；全文、空间、数组和 JSON 包含等需求则可能需要 GIN、GiST、全文或其他结构。

7.3 InnoDB 聚簇索引影响

表：

```
CREATE TABLE orders (
  id BIGINT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  status TINYINT NOT NULL,
  created_at DATETIME NOT NULL,
  KEY idx_user_time(user_id, created_at)
) ENGINE=InnoDB;
```

idx_user_time 叶子逻辑上包含：

```
(user_id, created_at, id)
```

查询二级索引未覆盖的列时：

```
secondary index -> get primary key -> clustered index lookup
```

主键越宽，所有二级索引越大。没有显式主键时 InnoDB 会寻找合适的唯一非空键，或生成隐藏聚簇键；生产表应显式定义主键。

7.4 PostgreSQL heap 与索引

PG 索引条目指向 (block, offset)。普通 Index Scan:

```
B-Tree -> TID -> heap page -> visibility check -> return tuple
```

因为 MVCC 可见性信息主要在 heap tuple, Index Only Scan 也不保证完全不访问 heap。只有 visibility map 显示页面 all-visible 时, 才可跳过 heap 可见性检查。

PG B-Tree 还可进行 deduplication 等内部优化; 但面试回答重点应是 heap/TID、MVCC 和 visibility map 的关系。

7.5 索引选择性与成本

低选择性列不意味着“绝对不能建索引”。是否使用取决于:

- 过滤后比例。
- 是否与其他列组成联合索引。
- 是否能避免排序或回表。
- 表大小、缓存命中和并发。
- PG 是否适合部分索引或 BRIN。

例如订单表 status 只有 5 个值, 单列索引价值可能低; 但 (status, created_at) 对“查询最近待处理订单”可能非常有效。

7.6 PG 特有的常见索引类型

- B-Tree: 等值、范围、排序。
- GIN: 一个值包含多个 key, 常用于数组、JSONB、全文。
- GiST: 通用搜索树, 常用于几何、range、最近邻等。
- BRIN: 保存块范围摘要, 适合超大且与物理顺序高度相关的数据, 如时间序列。
- Hash: 等值查询, 但 B-Tree 通常已足够, 需基于实际收益选择。

索引类型不是越“高级”越好。例如 GIN 写放大和维护成本通常高于 B-Tree, BRIN 对数据物理相关性差的列效果有限。

8. 联合索引、覆盖索引、部分索引与表达式索引如何设计？

8.1 结论先行

索引设计不是为单个 WHERE 条件服务，而是为完整访问路径服务：过滤、连接、排序、返回列和写入成本必须一起考虑。

联合索引列顺序常见原则：

1. 能缩小范围的等值条件与租户键优先。
2. 再放范围或排序列，使过滤和 ORDER BY 尽量复用同一顺序。
3. 覆盖列只为减少额外访问，不应无限堆积。
4. 使用实际查询分布和执行计划验证，不要机械套用“选择性最高放最左”。

8.2 最左前缀与范围

索引 (tenant_id, status, created_at) 可自然服务：

```
WHERE tenant_id = ?
WHERE tenant_id = ? AND status = ?
WHERE tenant_id = ? AND status = ? AND created_at >= ?
```

跳过 tenant_id 通常无法把整个索引当成 (status, created_at) 的有序树使用。优化器可能使用 skip scan、bitmap 组合等特殊路径，但不应作为核心设计前提。

“范围条件后的列完全不能使用”也过于绝对。后续列可能：

- 不能继续缩小连续索引扫描边界。
- 仍可在索引层过滤，例如 MySQL Index Condition Pushdown。
- 仍可作为覆盖列或帮助排序，具体取决于谓词和执行计划。

8.3 覆盖索引

MySQL 中，如果查询所需列都能从二级索引获得，就可避免回聚簇索引：

```
SELECT created_at, status
FROM orders
WHERE user_id = ?
ORDER BY created_at DESC
LIMIT 20;
```

索引 (user_id, created_at, status) 可能覆盖查询。注意 InnoDB 二级索引已隐含主键，不一定要重复把主键写入索引定义。

PG 可使用 **INCLUDE** 添加非键列：

```
CREATE INDEX idx_orders_user_time
ON orders(user_id, created_at DESC)
INCLUDE (status, amount);
```

INCLUDE 列不参与搜索顺序和唯一性判断，只用于覆盖。但 Index Only Scan 仍取决于页面 all-visible；写频繁表上收益可能低于预期。

8.4 表达式与函数索引

PG：

```
CREATE INDEX idx_user_email_lower ON app_user (lower(email));
SELECT * FROM app_user WHERE lower(email) = lower(?);
```

MySQL 8 支持函数索引，也可使用生成列：

```
ALTER TABLE app_user
ADD COLUMN email_norm VARCHAR(320)
GENERATED ALWAYS AS (lower(email)) STORED,
ADD UNIQUE INDEX uk_email_norm(email_norm);
```

查询表达式必须与索引表达式在语义上匹配。不要假设优化器能把任意等价表达式都改写成同一个索引条件。

8.5 PG 部分索引

```
CREATE INDEX idx_unpaid_order_time
ON orders(created_at)
WHERE status = 'UNPAID';
```

适合只查询少量活跃状态、软删除未删除记录、多租户中的特殊子集。优点是索引小、缓存友好、写入维护少。

限制：查询谓词必须能被 planner 证明蕴含索引谓词。过度参数化或写法不匹配，可能导致部分索引不被使用。

MySQL 没有同等通用的 partial index，可通过生成列、拆表或其他建模方式模拟特定场景。

8.6 不可见索引与在线验证

MySQL invisible index 可让优化器默认忽略某索引，但仍维护它，适合验证“删除索引是否影响计划”，比直接 DROP 风险低。

PG 可通过统计与计划工具评估索引；删除生产索引可使用 **DROP INDEX CONCURRENTLY** 降低阻塞，但有事务和语法限制。不要用禁用扫描参数作为长期方案，它更适合诊断 planner 选择。

8.7 索引过多的代价

- 每次 INSERT/UPDATE/DELETE 维护更多树。

- 增加 WAL/redo、复制流量、缓存占用和备份体积。
- 在线 DDL 和故障恢复更慢。
- 优化器搜索空间和统计维护增加。

索引治理应基于使用统计、慢 SQL 和业务 SLA。删除前要考虑月末、对账、灾备脚本等低频查询。

9. JOIN、子查询、CTE 和窗口函数如何选择？

9.1 结论先行

SQL 写法应优先表达正确语义，再让优化器选择物理计划。不要把“JOIN 一定比子查询快”“CTE 一定会物化”当成固定规则。现代 MySQL 与 PG 都会进行子查询展开、半连接、反连接、CTE 内联或物化等优化，但版本、统计和写法会影响结果。

9.2 JOIN 算法

常见物理算法：

Nested Loop

```
for each row in outer:
    find matching rows in inner
```

外表结果小、内表有好索引时非常高效。外表估算错误可能导致灾难性循环。

Hash Join

对较小输入构建哈希表，再扫描另一输入探测。适合等值连接，受 work memory 和溢写影响。PG 长期支持 Hash Join；MySQL 8 也支持 hash join，但具体计划要以 EXPLAIN 为准。

Merge Join

两个输入按连接键有序时线性合并，适合大结果和已有排序。PG 常见；MySQL 优化器的实现和展示方式不同，不能直接套 PG 术语分析所有计划。

9.3 EXISTS、IN 与 JOIN

判断存在性时优先表达存在语义：

```
SELECT o.*
FROM orders o
WHERE EXISTS (
    SELECT 1 FROM payment p
    WHERE p.order_id = o.id AND p.status = 'SUCCESS'
);
```

如果改成 JOIN，支付表一对多可能使订单重复，需要 DISTINCT，既改变语义又增加成本。

NOT IN 遇到 NULL 有三值逻辑陷阱：子查询只要包含 NULL，结果可能全部变为 UNKNOWN。反连接通常使用 NOT EXISTS 更直观：

```
WHERE NOT EXISTS (
  SELECT 1 FROM blacklist b WHERE b.user_id = u.id
)
```

9.4 CTE

CTE 提高复杂 SQL 可读性：

```
WITH paid AS (
  SELECT order_id, max(paid_at) AS paid_at
  FROM payment
  WHERE status = 'SUCCESS'
  GROUP BY order_id
)
SELECT ...
```

PG 现代版本可对非递归、无副作用、合适的 CTE 内联，也支持 **MATERIALIZED / NOT MATERIALIZED** 控制；旧版本常把 CTE 作为优化屏障。MySQL CTE 可能被合并或物化，取决于优化器。

物化不是天然坏事：当中间结果被多次复用或需要隔离昂贵计算时可能有利；但会阻止谓词下推并产生临时数据。

9.5 窗口函数

“每个用户最近一笔订单”：

```
SELECT *
FROM (
  SELECT o.*,
    row_number() OVER (
      PARTITION BY user_id ORDER BY created_at DESC, id DESC
    ) AS rn
  FROM orders o
) x
WHERE rn = 1;
```

窗口函数不会像 GROUP BY 那样丢失明细行，适合排名、累计、移动平均和组内 Top-N。但大范围排序可能消耗大量内存和临时空间，应配合过滤、索引和分区策略。

10. 分区表什么时候有价值，为什么不能把分区当万能优化？

10.1 结论先行

分区主要解决**数据生命周期**、**批量维护**和**裁剪扫描范围**，不是自动让所有查询变快。查询若不能使用分区键裁剪，可能访问大量分区，规划和执行反而更慢。

10.2 适合分区的场景

- 按时间保存海量日志、流水、审计数据，需要快速删除历史分区。
- 查询通常带时间范围或租户范围，可做 partition pruning。
- 维护操作希望按分区执行，例如归档、备份、重建索引。
- 单表/单索引过大，局部数据治理明显受益。

不适合：数据量小、查询不带分区键、分区数量失控、希望靠分区解决热点单行写入。

10.3 MySQL 分区重点

MySQL 原生分区把一张逻辑表拆成多个物理分区。优化器在能推导分区条件时执行 pruning。

常见陷阱：

- 唯一键通常必须包含所有分区列，影响主键设计。
- 分区过多增加元数据、打开表和维护成本。
- 分区内仍需要合适索引；分区不是索引替代品。
- 不要用表达式让 pruning 无法推导，需通过 EXPLAIN 验证访问分区。

10.4 PostgreSQL 声明式分区

PG 支持 range、list、hash 分区，分区本质是独立关系。可为不同分区设置索引、表空间和维护策略。

时间分区常见：

```
CREATE TABLE events (  
  tenant_id bigint NOT NULL,  
  occurred_at timestamptz NOT NULL,  
  payload jsonb NOT NULL  
) PARTITION BY RANGE (occurred_at);
```

删除历史数据可 DROP/DETACH PARTITION，远大于 DELETE 更快，也减少 WAL 和 vacuum 压力。

风险：

- 数千或数万分区增加 planner 和 catalog 压力。
- 全局唯一约束受分区键限制，需重新设计唯一性。
- 跨分区更新可能相当于 delete + insert。
- 必须自动创建未来分区并监控默认分区。

10.5 分区、分库分表与 Sharding

分区通常仍在同一个数据库实例内，不能解决单机 CPU、内存、IO 和故障域上限。Sharding 把数据分布到多个实例，能水平扩展，但带来跨分片事务、Join、全局唯一、重分片和运维复杂度。

面试中应明确：先优化单实例与数据模型，达到明确瓶颈后再分片；分片是架构复杂度交换容量，不是免费性能。

第三部分：执行计划与性能优化

11. 如何阅读 MySQL EXPLAIN ANALYZE 与 PostgreSQL EXPLAIN?

11.1 结论先行

执行计划的核心不是看字段，而是回答四个问题：

1. 从哪里读数据，读了多少？
2. 用什么 Join 顺序和算法？
3. 估算行数与实际行数是否严重偏离？
4. 时间花在扫描、循环、排序、聚合、锁等待还是 IO？

只看“是否使用索引”是不够的。一个走索引但随机回表百万次的计划，可能比顺序扫描更慢。

11.2 MySQL EXPLAIN

传统表格字段常见关注点：

- **type**: 访问类型，`const/ref/range/index/ALL` 等只表示访问方式，不直接等于性能等级。
- **possible_keys / key**: 候选与实际索引。
- **key_len**: 使用的索引键长度，可辅助判断联合索引使用列。
- **rows**: 估算扫描行数。
- **filtered**: 条件过滤比例估算。
- **Extra**: `Using index`、`Using index condition`、`Using temporary`、`Using filesort` 等。

`Using filesort` 不一定写磁盘，也不一定必须消除；它表示排序不能直接按所需索引顺序输出。小结果内存排序可能非常快。

MySQL 8 `EXPLAIN ANALYZE` 会真实执行语句并展示 iterator 的实际行数、循环和耗时。生产使用要注意：对 `UPDATE/DELETE` 或极重查询可能产生真实影响，应优先在只读副本、事务回滚环境或安全数据集验证。

示例关注：

```
-> Nested loop inner join
    -> Index range scan on o ... (actual rows=100 loops=1)
    -> Single-row index lookup on u ... (actual rows=1 loops=100)
```

总成本要考虑 $\text{actual rows} \times \text{loops}$ 。内层一次只读一行，但循环 100 万次仍可能很慢。

11.3 PostgreSQL EXPLAIN

推荐诊断格式：

```
EXPLAIN (ANALYZE, BUFFERS, WAL, VERBOSE, SETTINGS)
SELECT ...;
```

按需使用，避免对生产写语句直接执行。重点节点：

- Seq Scan：顺序扫描，不一定坏；读取大比例数据时可能最优。
- Index Scan：索引定位后访问 heap。
- Index Only Scan：理论覆盖，但看 Heap Fetches。
- Bitmap Index Scan + Bitmap Heap Scan：适合中等选择性，先收集 TID 再按页访问。
- Nested Loop / Hash Join / Merge Join。
- Sort：关注 method、memory、是否 external merge 写磁盘。
- Hash：关注 buckets、batches、memory，batches 增大常意味着溢写。
- Gather/Gather Merge：并行计划，关注实际启动 worker 数。

BUFFERS 可显示 shared hit/read/dirtied/written。大量 shared read 说明真实 IO；大量 hit 但仍慢，可能是 CPU、重复循环或锁等待。

11.4 估算与实际偏差

示例：

```
estimated rows=10, actual rows=500000
```

后果：优化器可能选择 Nested Loop，以为外层只有 10 行，实际循环 50 万次。

先查：

- 统计是否过期。
- 多列相关性是否未被表达。
- 数据是否倾斜，常量是否属于热点值。
- 隐式类型转换或函数是否影响选择性。
- 参数化计划是否复用了不适合当前参数的通用计划。

不要在没有证据时直接强制索引或关闭某种 Join。

11.5 MySQL 与 PG 计划字段不能机械对照

MySQL cost 与 PG cost 都是优化器内部相对成本，不是毫秒，且模型单位不同。不能用两个数据库的 cost 数值比较谁更快。

真正对比需要：

- 相同业务语义和数据分布。
- 预热与冷缓存分别测试。
- 并发而非只跑单连接。
- 观察 P95/P99、吞吐、CPU、IO、日志量和恢复影响。

11.6 面试题模板

拿到计划后可以这样说：

我先看根节点总耗时，再沿最耗时分支向下定位；同时比较 estimated rows 与 actual rows，检查 loops 放大。若是 PG 再看 BUFFERS 和临时文件，若是 MySQL 看访问行数、回表、临时表和排序。确认是估算问题、访问路径问题还是资源等待后，再决定补统计、改 SQL、建索引或调整内存，而不是看到全表扫描就直接加索引。

12. 优化器统计信息为什么会错误导致错误执行计划？

12.1 结论先行

成本优化器不实际执行所有候选计划，而是基于统计信息估算行数 and 成本。统计错误会层层放大，特别是 Join 顺序和算法。

统计问题常见来源：

- 数据快速变化，统计过期。
- 数据分布极度倾斜。
- 多列强相关，但优化器按独立性估算。
- 参数值差异巨大，通用计划不适合所有参数。
- 表达式、函数或自定义类型缺少统计。

12.2 MySQL 统计

InnoDB 会维护持久化统计，优化器可使用直方图改善无索引列或倾斜列的选择性估算：

```
ANALYZE TABLE orders
UPDATE HISTOGRAM ON status WITH 32 BUCKETS;
```

直方图不是越多越好：

- 构建和维护有成本。
- 数据变化后会过期。
- 有索引列已有索引统计，是否额外需要要验证。

MySQL 还可能受到 prepared statement、范围估算、索引 dive 和相关列的影响。执行计划突然变化时，应对比表统计更新时间、数据分布、版本和参数，而不是只怀疑“优化器抽风”。

12.3 PostgreSQL 统计

ANALYZE 收集每列的 null fraction、distinct、most common values、histogram、correlation 等。default_statistics_target 或列级 statistics target 控制样本细度。

多列相关性可用扩展统计：

```
CREATE STATISTICS st_orders_tenant_status
(dependencies, mcv, ndistinct)
ON tenant_id, status
FROM orders;
ANALYZE orders;
```

适合 `tenant_id` 与 `status` 强相关等情况。扩展统计不会自动解决所有 Join 相关性，仍需看计划。

12.4 参数敏感计划

同一 SQL：

```
WHERE tenant_id = $1 AND status = $2
```

小租户返回 10 行，大租户返回千万行，最佳计划完全不同。

PG prepared statement 可能在 custom plan 与 generic plan 之间权衡；MySQL 也会基于参数和执行阶段做优化。治理方式包括：

- 拆分极端业务路径。
- 改进统计和索引。
- 避免把高度不同的查询强行统一。
- 必要时对特定会话或语句调整计划策略，但先测量。

12.5 统计维护与自动化

- MySQL 监控表数据变化、统计重算和计划变化；必要时执行 `ANALYZE TABLE`，但大表操作要评估锁和资源。
- PG 依赖 `autovacuum` 的 `analyze`；高变更表应单独降低 `autovacuum_analyze_scale_factor` 或设置阈值。
- 统计更新后计划可能改变，发布与数据迁移时要做计划回归。

13. 一条慢 SQL 应如何系统优化？

13.1 结论先行

慢 SQL 优化顺序通常是：

```
确认业务目标和慢的边界
-> 获取 SQL、参数、频率和等待类型
-> 执行计划 + 实际行数
-> 数据分布与索引
-> 改写 SQL / 数据模型
-> 验证并发与回归
```

不要只拿一条脱敏 SQL 在空库上 EXPLAIN。参数、事务状态、缓存和并发都会影响结果。

13.2 第一步：确认“慢”的定义

必须知道：

- 平均慢还是 P99 慢？
- 数据库执行慢，还是连接池等待、网络、结果序列化慢？
- 单次 2 秒低频报表，还是每秒 5000 次的 20ms 点查？
- 慢发生在主库、只读副本还是特定租户？
- 是计划改变、数据增长还是锁等待？

高频小 SQL 的总资源消耗可能比单条大 SQL 更严重。

13.3 第二步：减少不必要工作

常见改写：

- 不使用 `SELECT *`，减少 heap/回表、网络和反序列化。
- 把过滤尽早下推，避免先 Join/聚合大集合。
- 用 `EXISTS` 表达存在性，避免一对多 JOIN + DISTINCT。
- 消除 N+1，但也不要构造巨大的 IN 列表和超宽 Join。
- 把可计算的常量放在参数侧，不对索引列做无谓函数。

错误：

```
WHERE date(created_at) = '2026-06-29'
```

更可索引：

```
WHERE created_at >= '2026-06-29 00:00:00'
AND created_at < '2026-06-30 00:00:00'
```

若业务长期按表达式查询，也可使用表达式/函数索引，但要评估写入成本。

13.4 隐式转换

字符串列与数字参数比较、不同 collation、不同字符类型连接，可能导致转换和索引失效或估算错误。

Java 层应使用正确 JDBC 类型，不要把所有参数 `setString`。跨库迁移尤其要检查 PG 更严格的类型转换规则。

13.5 索引设计

为查询：

```
SELECT id, amount, created_at
FROM orders
WHERE tenant_id = ?
AND status = 'PAID'
AND created_at >= ?
ORDER BY created_at DESC
LIMIT 50;
```

候选索引可能是：

```
(tenant_id, status, created_at DESC)
```

再考虑覆盖列。验证内容：

- 扫描行数是否显著下降。
- 是否避免排序。
- 是否因覆盖减少 heap fetch/回表。
- 对写入、存储和其他查询的影响。

13.6 避免一次处理过多数据

大 DELETE：

```
DELETE FROM events WHERE created_at < ?;
```

可能产生长事务、巨量 undo/WAL、复制延迟和锁压力。可采用：

- 按主键/时间小批删除并提交。
- 时间分区直接删除旧分区。
- 限速并监控副本延迟。

- 预估回收机制：MySQL purge 与 PG vacuum 并非提交后空间立即缩小。

13.7 验证优化

至少比较：

- 实际行数、loops、buffer read/hit。
- P50/P95/P99 和吞吐。
- CPU、IOPS、临时文件、redo/WAL 量。
- 并发锁等待。
- 冷热缓存。
- 对复制延迟和写入的影响。

优化后的 SQL 单次快了，但新索引导致写入 P99 和复制延迟恶化，也可能是失败优化。

14. 深分页、COUNT、ORDER BY 和 Top-N 应如何优化？

14.1 深分页

```
SELECT ...
FROM orders
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 1000000;
```

数据库仍需找到并丢弃前 100 万行。即使有索引，也会扫描大量条目；若需回表，成本更高。

使用 keyset/seek pagination：

```
SELECT id, created_at, amount
FROM orders
WHERE (created_at, id) < (?, ?)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

MySQL/PG 均支持行值比较，但要确认 NULL 和排序方向。游标必须包含唯一的稳定 tie-breaker，例如 id，否则同一时间值可能重复或漏数据。

14.2 分页一致性

并发插入时，OFFSET 页码天然可能漂移。seek pagination 更稳定，但仍需定义一致性需求：

- 普通信息流允许看到新数据并轻微变化。
- 导出/对账需要固定快照或任务表。
- 长时间保持数据库事务快照会阻碍 purge/vacuum，不应为大导出一直占用在线主库事务。

14.3 COUNT(*)

“PG COUNT(*) 很慢、MySQL 很快”是过度概括。

- InnoDB 不保存事务一致的精确总行数，COUNT(*) 也通常需要扫描某个索引。
- PG 需要按 MVCC 可见性判断行，精确 count 通常扫描表或索引。
- MyISAM 过去有快速总数，但不是当前 InnoDB 语境。

大表首页若只需展示“约 100 万条”，可以：

- 使用统计估算。
- 维护异步计数表或缓存。
- 只判断 has_next，多取一条，不返回总页数。
- 对有严格一致要求的计数接受真实成本或重新建模。

14.4 ORDER BY 与索引

索引能直接提供排序的前提是：过滤条件与索引顺序匹配，且排序方向和列顺序可被同一路径满足。

索引 (tenant_id, created_at DESC, id DESC) 可服务：

```
WHERE tenant_id = ?  
ORDER BY created_at DESC, id DESC  
LIMIT 50
```

但若跨大量 tenant 排序，或 ORDER BY amount，该索引无法直接提供目标顺序。

14.5 Top-N

小 LIMIT 的排序，数据库可使用 top-N heap 等算法而不完全排序所有数据；最优方案仍常是合适索引 + 早期过滤。

组内 Top-N 使用窗口函数直观，但全表窗口可能昂贵。对于极高频“每用户最近 10 条”，可考虑按 (user_id, created_at desc) 索引；对于预计算榜单，可使用汇总表或缓存，但需要明确更新一致性。

15. 大批量写入、在线 DDL 和索引构建如何控制风险？

15.1 批量写入

原则：减少往返、控制事务大小、避免压垮日志和副本。

MySQL:

- 多值 INSERT 或 JDBC batch。
- 导入可使用 **LOAD DATA**，但需审计格式、权限与复制影响。
- 单事务过大导致 undo、redo、锁和复制应用压力。

PostgreSQL:

- **COPY** 通常比逐行 INSERT 高效。
- JDBC batch/pipeline 可减少网络往返。
- 大事务会积压 WAL、阻碍 vacuum、放大复制延迟。

批量大小不能只看吞吐，应看 P99、WAL/redo、checkpoint、复制延迟、锁和失败重试成本。

15.2 Upsert

MySQL:

```
INSERT INTO inventory(sku_id, qty)
VALUES (?, ?)
ON DUPLICATE KEY UPDATE qty = VALUES(qty);
```

新版本推荐留意 **VALUES()** 相关语法演进，可使用别名写法。PG:

```
INSERT INTO inventory(sku_id, qty)
VALUES ($1, $2)
ON CONFLICT (sku_id)
DO UPDATE SET qty = EXCLUDED.qty;
```

Upsert 仍可能死锁，特别是批量中键顺序不同。对键排序、缩短事务并重试。

15.3 MySQL Online DDL

MySQL **ALTER TABLE** 可能选择 **INSTANT**、**INPLACE** 或 **COPY** 等算法，是否锁表取决于具体变更、版本和表结构。不能看到 **ONLINE** 就认为零影响。

上线前：

```
ALTER TABLE ... ALGORITHM=INSTANT;
-- 或显式声明期望算法/锁级别，使不满足时失败，而不是静默退化
```

监控 metadata lock。一个长期未提交事务可能阻塞 DDL，DDL 排队后又阻塞后续查询，形成事故链。

超大表可使用 gh-ost、pt-online-schema-change 等外部工具，但它们也会增加 binlog、触发器/影子表、复制和切换风险，必须演练。

15.4 PostgreSQL DDL 与索引

PG 很多 DDL 是事务性的，但锁强度可能很高。CREATE INDEX 普通模式会阻塞写；CREATE INDEX CONCURRENTLY 允许并发写，但：

- 需要多阶段扫描。
- 耗时和 IO 更高。
- 不能放在普通事务块中。
- 失败可能留下 invalid index。

添加带默认值的列、验证约束、重写数据等行为随版本优化而不同。安全做法是查官方版本语义，先在同量级副本演练并观察锁。

增加非空约束可采用分步策略：

1. 添加可空列或快速元数据变更。
2. 小批回填。
3. 添加 CHECK ... NOT VALID（PG 场景）。
4. 在线验证约束。
5. 最终设置 NOT NULL 或切换应用。

15.5 变更的通用护栏

- 设置合理 lock timeout，避免无限排队。
- 先检查长事务、空闲事务和复制延迟。
- 明确回滚方式；DDL 回滚不等于业务可无损回退。
- 限速、暂停开关、容量预留。
- 变更前后做执行计划和索引使用回归。
- 不在业务高峰首次验证。

第四部分：存储引擎、空间回收与连接治理

16. InnoDB Buffer Pool、redo、undo、doublewrite 与 checkpoint 如何协作？

16.1 结论先行

InnoDB 性能核心是“以内存页为工作集、用 redo 保证崩溃恢复、用 undo 支持事务与 MVCC、用 checkpoint 和后台刷脏控制恢复边界”。这些组件不是孤立参数，而是一条完整写入链路。

16.2 Buffer Pool

Buffer Pool 缓存数据页和索引页。读流程：

```
查 Buffer Pool
-> 命中：直接读取内存页
-> 未命中：从表空间读取页，放入 Buffer Pool
```

写流程通常先修改内存页，成为 dirty page，再由后台线程刷盘。Buffer Pool 大并不代表越大越好：还需给操作系统、连接、排序、Performance Schema、备份和其他进程留内存，避免 swap。

监控关注：

- buffer pool hit ratio 只能作为辅助，高命中不代表没有性能问题。
- dirty pages、free pages、page read/write rate。
- 数据集增长后是否发生持续淘汰。
- checkpoint age 与 redo 空间压力。

16.3 Redo Log

redo 记录页修改所需的恢复信息。提交时先保证 redo 达到持久化要求，脏页之后再刷。

redo 容量太小：checkpoint 频繁、刷脏激进、吞吐抖动。

redo 容量太大：通常有利于平滑写入，但异常恢复可能更久，磁盘和运维也需考虑。现代 MySQL 的 redo 配置方式与旧版本不同，不能只背旧的 `innodb_log_file_size × files`。

16.4 Undo 与 Purge

undo 用于：

- 事务回滚。
- MVCC consistent read 重建旧版本。

已提交不代表 undo 立即可删。只要存在可能读取旧版本的 Read View, purge 就必须保留相关 undo。

排查长事务：

```
SELECT trx_id, trx_started, trx_state, trx_rows_modified,
       trx_mysql_thread_id, trx_query
FROM information_schema.innodb_trx
ORDER BY trx_started;
```

还应从应用连接池和事务边界找到来源，不能只 kill 后不修复。

16.5 Doublewrite

数据库页通常大于底层原子写单元，断电可能产生 torn page：页只写了一部分。doublewrite 先把页写入一块安全区域并落盘，再写最终表空间位置；若最终页损坏，可从 doublewrite 副本恢复，再应用 redo。

它不是简单“所有数据写两遍所以一定慢一倍”。实际有批量、顺序化和存储优化。是否调整必须结合云盘原子写能力、版本实现和故障模型，不能为了基准测试轻易关闭。

16.6 Change Buffer

对不在 Buffer Pool 的非唯一二级索引页修改，InnoDB 可缓冲变化，后续页面读入时合并，减少随机 IO。唯一索引需要立即检查唯一性，不能同样延迟。

在 SSD、写密集、缓存大小不同的环境中收益不同，应看 change buffer 指标，不要默认越大越好。

16.7 Checkpoint 与刷脏

checkpoint 推进表示更早的 redo 对应修改已经安全写入数据文件，可以复用日志空间。刷脏太慢会逼近 redo 容量上限，触发同步或激进刷盘，表现为吞吐突然下降和 IO 飙升。

调优应联动：

- redo capacity。
- innodb_io_capacity 与真实存储能力。
- Buffer Pool dirty page 比例。
- 写入突发、批处理和 checkpoint age。

17. PostgreSQL VACUUM、autovacuum、HOT 与表膨胀如何理解？

17.1 结论先行

PG MVCC 把旧行版本保存在 heap，因此持续 UPDATE/DELETE 会产生 dead tuples。VACUUM 的主要职责是：

- 回收死元组空间供表内复用。
- 更新 visibility map，帮助 Index Only Scan。
- 冻结旧事务 ID，防止 transaction ID wraparound。
- 配合 ANALYZE 维护统计。

autovacuum 不是可选“清理工具”，而是 PostgreSQL 正常运行机制的一部分。

17.2 为什么 UPDATE 会产生膨胀

```
UPDATE row v1
-> v1 标记旧版本
-> 写入 v2
-> 索引可能新增指向 v2 的条目
-> v1 等旧快照结束后可回收
```

若更新频繁、长事务存在或 autovacuum 跟不上：

- heap dead tuples 增多。
- 索引保留无效条目。
- 扫描更多页，缓存命中下降。
- 表文件增长。

普通 VACUUM 主要让空间在表内复用；无法总是缩小操作系统文件。VACUUM FULL 重写表并强锁，通常需要维护窗口。可根据情况使用 `pg_repack` 等外部方案，但需评估额外空间、触发器和复制。

17.3 HOT Update

HOT 条件简化为：

- 更新没有改变任何索引所引用的列，包括表达式和部分索引涉及列。
- 同一 heap page 有足够空间容纳新 tuple。

HOT 避免创建普通新索引条目，新版本通过 heap 内链连接。提高 HOT 比例的手段：

- 不为频繁更新且低价值的列建索引。
- 为热点更新表设置适当 **fillfactor**，预留页内空间。
- 控制 tuple 宽度。

降低 fillfactor 会增加表初始体积，是以空间换更新效率，需实测。

17.4 Autovacuum 触发

常见触发思想：

```
threshold + scale_factor × table_size
```

大表使用默认 scale factor 时，需要积累非常多变化才触发，可能太晚；小而极热的表则可能频繁 vacuum。可按表设置：

```
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.02,
    autovacuum_analyze_scale_factor = 0.01
);
```

具体值必须根据变更率、每轮 vacuum 时间、IO 和 dead tuple 增长决定。

17.5 长事务与 idle in transaction

长事务持有旧快照，使 VACUUM 无法删除其可能看到的 tuple。尤其危险的是应用开启事务后长时间空闲：

```
SELECT pid, xact_start, state, wait_event_type, query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;
```

治理：

- 应用事务不跨远程调用和人工交互。
- 设置 **idle_in_transaction_session_timeout** 等保护参数，先评估兼容性。
- 大导出使用只读副本、批次和快照方案，不长期占用主库。

17.6 Freeze 与 XID Wraparound

PG 事务 ID 有有限空间并按环形比较。非常老的未冻结 tuple 若不处理，可能引发 wraparound 风险，数据库会启动更激进的 anti-wraparound vacuum，严重时影响业务。

必须监控数据库和表的 **age(datfrozenxid) / age(relfrozenxid)**，不能只监控磁盘和 QPS。

17.7 常用观测

```
SELECT schemaname, relname,  
       n_live_tup, n_dead_tup,  
       last_vacuum, last_autovacuum,  
       last_analyze, last_autoanalyze  
FROM pg_stat_user_tables  
ORDER BY n_dead_tup DESC;
```

注意统计是估算和累计视图，必要时结合 `pg_stat_progress_vacuum`、表大小、扩展工具和执行计划。

18. MySQL 线程模型、PG 进程模型与连接池如何设计？

18.1 结论先行

数据库连接不是免费资源。连接数要由“可并行工作的数据库容量”决定，而不是应用实例数乘一个随意的 100。

- MySQL 每连接通常由线程服务，现代实现会有线程缓存等机制。
- PG 经典模型每连接一个 backend 进程，连接的内存和进程调度成本更明显。
- 连接池只能复用连接，不能创造数据库容量；过大的连接池会把排队从应用层转移到数据库内部，导致上下文切换、内存争抢和 P99 恶化。

18.2 连接池容量

估算起点：

```
数据库能稳定并发执行的活跃查询数
÷ 应用实例数
+ 少量余量
```

而不是：

```
最大 QPS × 平均延迟
```

后者得到的是系统并发需求，但数据库可能承受不了，必须在应用层排队、缓存或限流。

例如数据库在 64 个活跃查询时吞吐最佳，8 个应用实例每个池 50，总计 400 个连接会造成过载。可能更合理的是每实例 8-12，并设置获取超时与舱壁隔离。

18.3 MySQL 连接治理

关注：

```
SHOW GLOBAL STATUS LIKE 'Threads_connected';
SHOW GLOBAL STATUS LIKE 'Threads_running';
SHOW GLOBAL STATUS LIKE 'Connections';
SHOW GLOBAL STATUS LIKE 'Aborted_connects';
```

Threads_connected 很高但 Threads_running 很低，可能只是大量空闲池连接；真正并发压力更应看 running、CPU、锁和队列。

max_connections 不能作为性能调优按钮无限增大。每连接缓冲区如 sort/join/read buffer 可能按需分配，极端并发会放大内存。

18.4 PostgreSQL 与 PgBouncer

PG 常用 PgBouncer:

- session pooling: 客户端会话长期绑定服务端连接, 兼容性最好。
- transaction pooling: 每个事务借一个后端连接, 复用率高, 但 session 级状态、临时表、某些 prepared statement 和 advisory lock 使用受限。
- statement pooling: 限制最多, 较少用于通用事务应用。

使用 transaction pooling 时, 应用必须把事务边界写清楚, 不能依赖连接级临时状态。还要考虑 SET、search_path、临时表、LISTEN/NOTIFY 等特性。

18.5 Little's Law 与超时

近似:

$$\text{并发数 } L = \text{吞吐 } \lambda \times \text{平均响应时间 } W$$

QPS 2000、数据库平均 20ms, 平均在途约 40; 但 P99、突发和事务多语句会要求余量。连接池应有:

- 获取连接超时。
- SQL/statement timeout。
- 事务超时或业务 deadline。
- 失败快速返回与限流。

超时必须从外到内递减或一致规划, 避免应用已放弃请求, 数据库仍执行数分钟。

18.6 读写连接池隔离

可为核心交易、后台批处理、只读查询设置不同池和数据库用户:

- 核心池小而稳定, 优先保障。
- 报表池限制并发和超时。
- 只读池连接副本, 但必须处理复制延迟和一致性。

连接池隔离是舱壁, 不是把总连接数翻倍。

第五部分：复制、备份与高可用

19. MySQL 主从复制、GTID、半同步和 Group Replication 如何理解？

19.1 结论先行

MySQL 传统复制基于 binlog。源库提交事务后，副本接收 binlog 写入 relay log，再由 applier 执行。默认异步复制意味着源库返回成功时，事务可能尚未到达副本；源库突然永久丢失可能丢最近事务。

GTID 解决“事务身份和切换定位”问题，不自动保证零丢失。半同步降低事务未到任何副本的窗口，但不等于副本已应用，也不等于强一致集群。

19.2 复制链路

```
Source commit
-> binlog
-> dump sender
-> replica receiver
-> relay log
-> coordinator / worker apply
-> replica data
```

延迟可能发生在：

- 网络接收慢。
- relay log 写入受限。
- 单个大事务。
- 副本并行应用不足或冲突。
- 副本存在慢查询，与 apply 争抢 IO/CPU/锁。
- DDL 或热点行串行化。

Seconds_Behind_Source 不是完整延迟指标。还应看接收位置、执行位置、GTID 集合、Performance Schema 复制表和业务时间戳。

19.3 Row、Statement 与 Mixed

生产常用 row-based：记录行变化，复制确定性更好，适合 CDC；代价是大批量更新可能产生大量 binlog。

statement-based 日志小，但非确定函数、执行环境差异、锁和语义更复杂。mixed 会自动选择。选型还要考虑审计、CDC、恢复与版本兼容。

19.4 GTID

每个事务有全局唯一标识，副本记录已执行集合。故障切换时可基于“谁拥有最新 GTID 集合”选择候选，减少文件名/position 管理。

GTID 不能解决：

- 业务误删已经复制到所有节点。
- 异步复制未到达任何副本的数据丢失。
- 自动切换中的脑裂和路由错误。

19.5 半同步复制

半同步通常要求至少一个副本确认收到事务日志到某个持久化阶段后，源库才向客户端返回。关键点：

- “收到/写 relay log” 通常不等于 “已经执行并可查询”。
- 超时后可能退化为异步，需要监控状态。
- 网络延迟进入提交路径，P99 增加。
- 故障切换仍需选择包含最新事务的副本。

19.6 并行复制

多线程 applier 可提高吞吐，但事务之间的依赖、热点键和大事务会限制并行。优化源库批次、避免巨型事务、配置合适 worker，并监控每个 worker backlog。

不要用提高副本硬件掩盖源库持续产生不可并行的大事务。

19.7 Group Replication / InnoDB Cluster

Group Replication 使用组成员、复制协议和冲突认证，可支持单主或多主模式。InnoDB Cluster 通常配合 MySQL Shell 和 Router 管理拓扑与路由。

它降低自建切换复杂度，但并非“无需理解一致性”：

- 多主可能有写冲突和认证失败。
- 节点失去多数派时的行为需要明确。
- 跨地域延迟影响提交和可用性。
- 仍需独立备份，应对逻辑错误。

20. PostgreSQL 流复制、同步复制、逻辑复制和 replication slot 如何理解？

20.1 结论先行

PG 物理流复制发送 WAL，standby 重放相同物理变化，适合同版本/兼容版本的高可用和只读副本。逻辑复制发送表级逻辑变化，适合选择性复制、升级迁移和事件分发，但不自动复制所有 DDL、sequence 状态和数据库对象。

Replication slot 能防止所需 WAL 被过早删除，但消费者停滞时会导致主库 WAL 无限积压，必须监控和限额。

20.2 物理流复制

```
Primary WAL
-> WAL sender
-> network
-> WAL receiver
-> standby WAL
-> startup process replay
```

可分为：

- write lag：到达并写入副本 OS/文件系统。
- flush lag：副本持久化。
- replay lag：副本已应用并可供查询。

只看时间延迟不够，应看 LSN 字节差：

```
SELECT application_name, state, sync_state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS replay_bytes,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

20.3 同步复制

synchronous_commit 与 synchronous_standby_names 共同决定提交等待哪些副本阶段。可配置等待 remote write、flush 或 apply 等语义，具体值以版本文档为准。

权衡：

- 更强持久性或读后写一致性意味着网络和副本性能进入主库提交路径。
- 同步副本不可用时可能阻塞提交，需设计候选集合和故障切换。
- 跨地域同步会显著增加事务延迟。

20.4 Hot Standby 冲突

standby 重放 WAL 时，主库已经清理的旧版本可能仍被副本长查询需要，产生 recovery conflict。数据库可能取消副本查询以继续重放。

hot_standby_feedback 可减少清理冲突，但会把副本长查询影响传回主库，导致主库 dead tuples/bloat。
max_standby_streaming_delay 控制重放等待，但延迟和查询可用性需权衡。

20.5 逻辑复制

逻辑复制典型对象：publication 和 subscription。

```
-- publisher
CREATE PUBLICATION app_pub FOR TABLE orders, order_item;

-- subscriber
CREATE SUBSCRIPTION app_sub
CONNECTION '...'
PUBLICATION app_pub;
```

注意：

- 表需要合适 replica identity，UPDATE/DELETE 必须能标识行。
- DDL、sequence 和大对象等不一定自动同步。
- 初始快照与增量衔接需监控。
- 订阅端冲突可能使 apply 停止。
- 逻辑复制通常不提供跨表事务之外的任意重新排序，但消费者下游仍需幂等。

20.6 Replication Slot 风险

slot 记录消费者仍需要的 WAL 位置。消费者宕机或订阅失效时，主库不能删除相关 WAL，可能填满磁盘。

监控：

```
SELECT slot_name, slot_type, active,
       restart_lsn, confirmed_flush_lsn,
       pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS retained_bytes
FROM pg_replication_slots;
```

应设置告警、容量上限策略和停用流程。删除 slot 前确认消费者是否还能重建，否则会造成增量链断裂。

21. 备份、恢复与 PITR 应如何设计并验证？

21.1 结论先行

复制不是备份。误删、错误 UPDATE、勒索加密和应用逻辑错误会快速复制到所有副本。完整方案需要：

- 基础全量备份。
- 连续日志归档（binlog/WAL）。
- 异地、不可变或隔离存储。
- 明确保留策略和加密。
- 定期真实恢复演练。

备份成功的唯一可信标准是：在目标 RTO 内恢复成功，并验证数据与应用可用。

21.2 RPO 与 RTO

- RPO：最多能接受丢多少数据，例如 5 分钟。
- RTO：故障后多久恢复服务，例如 30 分钟。

设计前必须明确：

- 单表误删恢复，还是整库/整机灾难。
- 恢复到原集群还是隔离环境。
- 需要哪个时间点，如何确认错误发生边界。
- 备份体积、网络和解密速度是否满足 RTO。

21.3 MySQL 备份

逻辑备份：mysqldump、MySQL Shell dump utilities 等，优点是可读、可选择对象、跨版本灵活；大库恢复慢，索引重建耗时。

物理备份：复制数据文件和日志的一致快照，通常恢复更快；工具与版本兼容要求更高。

PITR：

```
恢复最近全量备份
-> 按顺序应用后续 binlog
-> 在目标时间/GTID/position 前停止
-> 校验并切换
```

必须保留 binlog 与备份的一致起点信息。expire_logs_days 等旧参数和现代 binlog expiration 配置要按版本确认。

21.4 PostgreSQL 备份

逻辑：

- `pg_dump`：单数据库/对象级逻辑备份。
- `pg_dumpall`：全局对象等，但大库通常更常组合使用。

物理：

- `pg_basebackup` 或成熟备份工具创建 base backup。
- 持续归档 WAL，支持 PITR。
- 现代 PG 支持增量 base backup 相关能力，但恢复链和工具流程必须按版本验证。

PITR：

```
恢复 base backup
-> 配置 restore_command 获取连续 WAL
-> 设置 recovery_target_time / LSN / name
-> 重放到目标点并停止/提升
```

缺少任意一段必要 WAL 都可能导致恢复失败。

21.5 恢复演练清单

- 随机选择备份，不只恢复最新一份。
- 在隔离网络恢复，避免误连生产。
- 校验校验和、行数、关键业务余额和外键。
- 启动应用只读验收，执行核心查询。
- 测量下载、解压、恢复、重放、重建索引和切换时间。
- 验证密钥、账号、对象存储权限和文档。
- 记录实际 RPO/RTO，而不是理论值。

21.6 常见失败

- 只备份数据，不备份用户、权限、参数和扩展。
- WAL/binlog 保留时间小于基础备份周期。
- 备份与生产在同一账号、同一区域，故障一起丢失。
- 从未恢复过，真正事故时才发现版本或插件不兼容。
- 逻辑备份恢复顺序错误，约束和触发器造成极慢导入。

22. 高可用切换如何避免数据丢失、脑裂和读到旧数据？

22.1 结论先行

高可用不是“有一个副本 + 自动切换”这么简单。完整方案必须回答：

1. 故障如何被可靠检测？
2. 谁拥有提升主库的唯一决策权？
3. 如何 fencing 旧主，避免双主写入？
4. 候选副本缺多少日志，能否接受数据丢失？
5. 客户端如何切换、重连和重试？
6. 故障恢复后旧主如何重新加入？

自动化越强，越需要清晰的多数派、租约或外部协调机制。

22.2 数据丢失窗口

异步复制中：

T1 在主库提交成功
-> 尚未发送或持久化到副本
-> 主库永久损坏

提升副本后 T1 丢失。半同步/同步复制可以缩小窗口，但增加写延迟和故障时阻塞风险。

选择候选节点时应比较：

- MySQL GTID executed / received 集合、relay log 和应用位置。
- PG receive_lsn、flush_lsn、replay_lsn 与 timeline。

不应只按“延迟秒数最小”选主。

22.3 脑裂与 fencing

网络分区时，旧主可能仍运行，只是管理系统访问不到。如果新主被提升而旧主继续接受写，就产生脑裂。

fencing 方式：

- 云 API/虚拟化层强制关机或隔离旧主。
- 存储层撤销旧主访问。
- 数据库管理组件通过多数派确保只有一方获得领导权。
- 路由层切断旧主写流量，但仅改 DNS 通常不够，因为旧连接仍存在。

切换流程应先确保旧主不可写，再开放新主流量。无法 fencing 时宁可暂停写入，也不要盲目双主。

22.4 Read-after-write

主库写成功后立即去异步副本读，可能读不到新数据。解决：

- 写后一定时间或同一会话读主库。
- 携带一致性 token：MySQL 等待 GTID，PG 等待 replay LSN 达到写入位置。
- 对强一致接口只读主库，对列表/推荐允许副本延迟。
- 使用同步 apply 语义，但提交延迟会增加。

不能通过“sleep 100ms”可靠解决，因为延迟可能超过固定时间。

22.5 客户端切换

应用侧需要：

- 连接获取和 SQL 超时。
- 对网络断开、只读错误、连接重置做有限重试。
- 事务级重试，而不是只重发最后一条 SQL。
- 幂等请求和唯一业务键，避免提交成功但响应丢失导致重复写。
- 连接池快速淘汰旧连接。

DNS TTL、代理缓存、JVM DNS 缓存和已有 TCP 连接都会影响切换速度。数据库 Router/Proxy、VIP 或服务发现需演练。

22.6 切回与时间线

PG 提升 standby 会产生新 timeline，旧主不能简单启动后继续作为主；通常需要 `pg_rewind` 或从新主重新克隆。MySQL 也需要确认旧主多出的 divergent transactions，不能直接接回造成冲突。

高可用演练必须包含 failback，而不仅是 failover。

第六部分：故障排查、迁移与系统设计

23. 数据库 CPU、IO、锁等待或延迟突然升高时如何排查？

23.1 总体方法

先分清数据库是“正在做大量有效工作”，还是“在等待”。建立时间线：

```
应用延迟/QPS/错误率
数据库连接/活跃会话
CPU、load、IO latency、磁盘空间、网络
慢 SQL、执行计划、锁等待
checkpoint、WAL/redo、复制、vacuum/purge
最近发布、DDL、批任务和数据增长
```

不要先重启。重启会丢失现场、清空缓存，并可能让恢复和缓存预热造成第二次冲击。

23.2 CPU 高

常见原因：

- 新 SQL 或计划退化，扫描行数暴增。
- 高频小查询/N+1。
- 排序、Hash、JSON/正则等 CPU 密集计算。
- 连接过多导致上下文切换。
- PG autovacuum 或并行查询；MySQL purge/刷脏等后台活动。

MySQL：

```
SELECT DIGEST_TEXT, COUNT_STAR,
       SUM_TIMER_WAIT/1e12 AS total_s,
       AVG_TIMER_WAIT/1e9 AS avg_ms,
       SUM_ROWS_EXAMINED, SUM_ROWS_SENT
FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 20;
```

PostgreSQL（需启用 pg_stat_statements）：

```
SELECT queryid, calls, total_exec_time, mean_exec_time,
       rows, shared_blks_hit, shared_blks_read,
       temp_blks_read, temp_blks_written, query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

总耗时 Top 与单次耗时 Top 都要看。

23.3 IO 延迟高

确认是读、写、fsync 还是临时文件：

- Buffer/cache 工作集失效，随机读上升。
- checkpoint 或批量写导致刷盘峰值。
- 排序/Hash 溢写临时磁盘。
- 备份、快照、重建索引争抢 IO。
- 云盘达到 IOPS/吞吐额度或 burst credits 用尽。
- WAL/redo 与数据文件共享瓶颈。

操作系统：

```
iostat -x 1
vmstat 1
pidstat -dru -p <pid> 1
```

关注设备 await、队列、util、吞吐和 CPU iowait，但云盘虚拟化环境要结合厂商指标。

PG 计划中 **BUFFERS** 和 temp read/write 很关键；MySQL 看 Performance Schema file IO、临时表和 InnoDB metrics。

23.4 锁等待

症状：CPU 不一定高，连接和响应时间持续增加。

MySQL：

```
SELECT waiting_pid, blocking_pid,
       waiting_query, blocking_query
FROM sys.innodb_lock_waits;
```

PostgreSQL：

```
SELECT a.pid AS waiting_pid,
       pg_blocking_pids(a.pid) AS blockers,
       a.wait_event_type, a.wait_event,
       now() - a.query_start AS wait_time,
       a.query
FROM pg_stat_activity a
WHERE cardinality(pg_blocking_pids(a.pid)) > 0;
```

找到最上游 blocker，而不是只 kill 大量下游等待者。检查 blocker 是否：

- 长事务/idle in transaction。
- DDL metadata lock。
- 应用事务内远程调用。

- 缺索引导致锁范围大。

23.5 连接耗尽

连接池报 timeout 不一定是数据库 max_connections 小，可能是 SQL 变慢后连接长期占用。

排查：

- 活跃 vs 空闲连接。
- 事务持续时间。
- 应用池 active/pending/creation rate。
- 数据库 wait event。
- 是否连接泄漏。

直接增加 pool 或 max_connections 常会加重雪崩。先限流、终止异常批任务、隔离非核心流量，再修 SQL/锁。

23.6 复制延迟

检查是 receive lag 还是 apply lag：

- 网络/源发送。
- 大事务在副本串行应用。
- 副本查询阻塞重放或争资源。
- DDL、唯一冲突、磁盘慢。
- PG slot/WAL；MySQL relay/applier worker。

止损可暂停报表、拆大事务、提升副本 IO、调整并行应用；不要跳过事务或随意重建副本掩盖数据一致性问题。

23.7 磁盘即将满

常见增长源：

- MySQL binlog/relay log/undo/临时文件/表空间。
- PG pg_wal 因 slot、归档失败或副本离线积压；表和索引膨胀；临时文件。
- 备份文件误放数据盘。

先确认文件类型和保留原因。直接删除 pg_wal、redo、数据文件是灾难性操作。应修复归档/slot/复制，再通过数据库支持的方式释放。

23.8 事故处理闭环

1. 保存指标、日志、计划和锁图。
2. 止损：限流、关闭批任务、切换只读、终止明确异常会话。
3. 根因：SQL、统计、索引、事务、参数或基础设施。
4. 修复：灰度、回归、容量验证。
5. 防复发：SLO、告警、变更审计、慢 SQL 基线、故障演练。

24. MySQL 与 PostgreSQL 如何做不停机迁移？

24.1 结论先行

跨数据库迁移不是简单导出导入。最难的是：

- 类型与 SQL 语义兼容。
- 全量与增量无缝衔接。
- 校验一致性。
- 切换窗口中的双写和回滚。

推荐“评估 → 改造 → 全量 → CDC 增量 → 影子验证 → 短暂停写/双写收敛 → 切换 → 观察 → 下线”的阶段化方案。

24.2 兼容性清单

数据类型：

- MySQL TINYINT(1) 与 PG boolean 不完全等价。
- UNSIGNED 在 PG 无直接同名类型，需扩大类型或 CHECK。
- DATETIME/TIMESTAMP 与 PG timestamp/timestamptz 语义不同。
- ENUM 可映射 PG enum、lookup 表或 varchar + check。
- JSON 函数、数组、空间类型需重写。

SQL：

- 反引号、大小写与标识符。
- AUTO_INCREMENT vs identity/sequence。
- ON DUPLICATE KEY UPDATE vs ON CONFLICT。
- GROUP BY 宽松模式差异。
- 空字符串、NULL、布尔和隐式转换。
- 锁和隔离级别默认值。

24.3 全量加载

过程：

1. 建目标 schema、约束和必要扩展。
2. 选择一致快照起点，并记录 binlog/GTID 或 LSN。

3. 并行导出和加载，控制目标 WAL/redo 与索引成本。
4. 大量数据可先加载再建部分二级索引，但主键/CDC 所需约束要提前设计。
5. 更新 sequence/identity 起始值。
6. 收集统计信息。

PG 导入后执行 ANALYZE；MySQL 检查统计和执行计划。

24.4 增量 CDC

常用 Debezium、云迁移服务或专用同步工具，从 MySQL binlog / PG logical decoding 捕获变更。

必须处理：

- DDL 变更冻结或双边兼容。
- 事务顺序与同键顺序。
- delete、主键更新、无主键表。
- 大事务和快照期间的日志保留。
- 消费幂等和重放。
- CDC 延迟与失败告警。

24.5 数据校验

不能只比较总行数。分层校验：

- 表级行数和聚合。
- 按主键范围分桶 count/sum/min/max。
- 稳定序列化后的 hash/checksum。
- 随机抽样业务对象。
- 关键不变量：账户余额、订单项合计、状态机合法性。
- 应用影子流量比较查询结果。

浮点、时间精度、字符排序和 JSON key 顺序可能造成“格式不同但语义相同”，校验程序需规范化。

24.6 双写的风险

应用双写两个数据库会遇到：A 成功 B 失败，没有天然原子性。更稳的做法：

- 单一主写 + Outbox/CDC 复制。
- 双写仅作为短期切换桥梁，并有重试、对账和补偿。
- 切换时设置写入栅栏，确保旧库最后位点已追平。

24.7 切换与回滚

切换步骤示例：

1. 停止 schema 变更和非必要批任务。
2. 进入短暂只读或通过写栅栏记录最后位点。
3. 等待 CDC lag 为 0，并做最终校验。
4. 切换连接和路由。
5. 开放小比例写入，观察错误、计划、锁和延迟。
6. 全量放开。

回滚难点：新库切换后产生的新写入如何同步回旧库。必须提前设计反向 CDC 或明确“切换点后只能向前修复”，不能只说“改回连接串”。

25. 场景设计：构建高并发订单与账户数据库

25.1 题目

设计一个电商订单与余额支付系统：

- 峰值下单 50,000 QPS，支付回调可能重复。
- 订单按用户查询最近记录，商家按状态扫描待处理订单。
- 余额不能超扣，订单状态不能逆向跳转。
- 数据保留 7 年，需要备份、跨可用区容灾和在线扩容。
- 允许订单列表秒级最终一致，但余额与支付结果要求强一致。

请分别说明 MySQL 或 PostgreSQL 下的数据模型、事务、索引、复制、分片和灾备设计。

25.2 需求分级

先按一致性分类：

强一致核心

- 账户余额扣减。
- 支付幂等。
- 订单状态机。
- 账务流水完整性。

可最终一致查询

- 用户订单列表缓存。
- 搜索、报表、风控特征。
- 商家运营统计。

不要让一个跨多个远程服务的大事务承担所有功能。

25.3 数据模型

```
account(  
  account_id bigint primary key,  
  balance decimal(20,2) not null,  
  version bigint not null,  
  updated_at ...  
)  
  
account_ledger(  
  account_id bigint foreign key references account(  
    account_id  
  ),  
  ledger_id bigint primary key,  
  amount decimal(20,2) not null,  
  created_at timestamp not null,  
  updated_at timestamp not null
```

```

    ledger_id bigint primary key,
    account_id bigint not null,
    business_type varchar(...),
    business_id varchar(...),
    amount decimal(20,2) not null,
    created_at ...,
    unique(account_id, business_type, business_id)
)

orders(
    order_id bigint primary key,
    user_id bigint not null,
    merchant_id bigint not null,
    status varchar(...),
    amount decimal(20,2) not null,
    created_at ...,
    updated_at ...,
    version bigint not null
)

payment_event(
    provider varchar(...),
    provider_event_id varchar(...),
    order_id bigint not null,
    payload ...,
    created_at ...,
    primary key(provider, provider_event_id)
)

outbox_event(
    event_id bigint primary key,
    aggregate_type varchar(...),
    aggregate_id varchar(...),
    event_type varchar(...),
    payload ...,
    created_at ...,
    published_at ...
)

```

关键点：

- 支付回调用供应商事件 ID 唯一约束幂等。
- 账务流水使用业务幂等键，余额与流水同一事务提交。
- Outbox 与业务变更同一数据库事务，异步发布到 MQ。
- 状态字段用 CHECK/应用状态机共同保护。PG 可使用更强约束能力；MySQL 8 也支持 CHECK，但仍需关注版本与 SQL mode。

25.4 余额扣减事务

单条条件更新：

```

UPDATE account
SET balance = balance - :amount,
    version = version + 1,

```

```
updated_at = CURRENT_TIMESTAMP
WHERE account_id = :account_id
AND balance >= :amount;
```

受影响行数为 1 才继续插入流水；否则余额不足。事务内：

1. 插入幂等流水或先检查唯一键。
2. 条件扣减余额。
3. 更新订单支付状态，带状态前置条件。
4. 插入 outbox。
5. 提交。

注意幂等流水与更新顺序要设计死锁访问顺序。重复请求命中唯一约束后读取原结果返回。

25.5 订单状态机

不要：

```
UPDATE orders SET status = 'PAID' WHERE order_id = ?;
```

应带前置状态：

```
UPDATE orders
SET status = 'PAID', version = version + 1
WHERE order_id = ?
AND status = 'PENDING_PAYMENT';
```

根据受影响行数区分成功、重复和非法状态。复杂状态可使用状态转换表或服务层集中校验，并保留状态流水。

25.6 索引

用户最近订单：

```
(user_id, created_at DESC, order_id DESC)
```

商家待处理：

```
(merchant_id, status, created_at, order_id)
```

PG 若待处理仅占很小比例，可用部分索引：

```
CREATE INDEX idx_orders_merchant_pending
ON orders(merchant_id, created_at, order_id)
WHERE status IN ('PAID', 'READY_TO_SHIP');
```

MySQL 可使用状态联合索引或生成列模拟活跃子集。所有索引需按真实写入成本评估。

25.7 分库分表

50,000 QPS 是否必须分片取决于单库基准、事务比例、数据量和硬件。若需要：

- 订单按 `user_id` 或 `order_id` 哈希分片，取决于主查询路径。
- 账户按 `account_id` 分片。
- 避免一个事务跨账户分片；转账业务需专门账务架构和分布式一致性设计。
- 商家维度跨分片查询可建立异步商家订单视图/搜索索引，而不是跨所有分片实时排序。

全局 ID 使用趋势递增的分布式 ID，并在 ID 中包含路由信息时要评估耦合和信息泄露。

25.8 缓存与读副本

- 余额不以缓存作为最终判断，强一致读写主库。
- 订单详情可 Cache Aside，更新数据库后删除缓存，并用事件修复。
- 列表可以从只读副本或查询服务读，明确秒级延迟。
- 支付完成跳转页需要 read-after-write，可读主库或等待复制 token。

25.9 高可用与灾备

同城多可用区：

- MySQL 可用半同步/Group Replication 或成熟云 HA。
- PG 可用同步流复制 + Patroni/云 HA 等管理组件。
- 写入 SLA 决定同步副本数量和超时退化策略。

异地灾备通常异步，RPO 不为 0。配合：

- 全量备份 + binlog/WAL 归档。
- 跨区域不可变存储。
- 每季度恢复与切换演练。
- 账务对账和业务补偿，处理极端丢失窗口。

25.10 容量与降级

容量估算至少包括：

```

每日新增行数 × 平均行宽
+ 索引放大
+ MVCC/页空洞
+ redo/WAL/binlog
+ 副本数量
+ 备份与保留
  
```

高峰降级：

- 网关限流与队列削峰。
- 非核心列表、推荐和统计降级。
- 批处理暂停。
- 核心数据库连接池舱壁。
- 数据库过载时快速失败，不无限堆积连接。

25.11 MySQL 与 PG 选型结论

该系统两者都能实现。若团队已有成熟 MySQL 分片和运维体系，MySQL 通常是低风险选择；若强依赖复杂约束、部分索引、丰富 SQL 和统一数据平台，PG 可能减少应用层复杂度。

最终决策需基于：

- 真实数据和并发基准。
- 故障恢复与备份演练。
- 团队运维和人才。
- 中间件、云服务与合规。
- 五年总成本，而不是单条 SQL 跑分。

附录 A：MySQL 与 PostgreSQL 高频对比速查

问题	MySQL / InnoDB	PostgreSQL
默认隔离级别	REPEATABLE READ	READ COMMITTED
表组织	聚簇索引	Heap + index
MVCC 旧版本	Undo	Heap old tuples
旧版本回收	Purge	VACUUM
提交日志	Redo + binlog 两阶段协调	WAL
复制	Binlog 逻辑复制为主	WAL 物理流复制 + 逻辑复制
覆盖访问	二级索引覆盖避免回聚簇索引	Index Only Scan + visibility map
部分索引	无通用 partial index	原生支持
表达式索引	支持函数索引/生成列	原生表达式索引
在线索引	依 DDL 算法和锁级别	CREATE INDEX CONCURRENTLY
连接模型	线程	进程，常配 PgBouncer
备份 PITR	物理/逻辑 + binlog	base backup + WAL archive
空间回收	purge 后页内复用，表空间缩小需条件/重建	vacuum 页内复用，缩小常需重写
JSON	JSON + 生成列/函数/多值索引	JSONB + GIN/表达式索引

附录 B：常用排障 SQL

B.1 MySQL

当前会话与事务：

```
SHOW FULL PROCESSLIST;

SELECT trx_id, trx_state, trx_started,
       trx_wait_started, trx_rows_locked,
       trx_rows_modified, trx_mysql_thread_id, trx_query
FROM information_schema.innodb_trx
ORDER BY trx_started;
```

锁等待：

```
SELECT * FROM sys.innodb_lock_waits;
SELECT * FROM performance_schema.data_lock_waits;
```

高成本 SQL：

```
SELECT DIGEST_TEXT, COUNT_STAR,
       SUM_TIMER_WAIT/1e12 total_s,
       AVG_TIMER_WAIT/1e9 avg_ms,
       SUM_ROWS_EXAMINED, SUM_ROWS_SENT,
       SUM_CREATED_TMP_DISK_TABLES
FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 30;
```

复制：

```
SHOW REPLICA STATUS\G
SELECT * FROM performance_schema.replication_applier_status_by_worker;
```

InnoDB：

```
SHOW ENGINE INNODB STATUS\G
SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool%';
SHOW GLOBAL STATUS LIKE 'Innodb_os_log%';
```

B.2 PostgreSQL

活动与等待：

```
SELECT pid, username, application_name, client_addr,
       state, xact_start, query_start,
       wait_event_type, wait_event,
       pg_blocking_pids(pid) blockers,
       query
FROM pg_stat_activity
```

```
WHERE datname = current_database()
ORDER BY xact_start NULLS LAST, query_start;
```

表与 vacuum:

```
SELECT schemaname, relname,
       n_live_tup, n_dead_tup,
       seq_scan, idx_scan,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

索引:

```
SELECT schemaname, relname, indexrelname,
       idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan;
```

复制:

```
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

Slot:

```
SELECT slot_name, slot_type, active,
       restart_lsn, confirmed_flush_lsn,
       pg_size_pretty(
         pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
       ) retained
FROM pg_replication_slots;
```

空间:

```
SELECT n.nspname, c.relname,
       pg_size_pretty(pg_total_relation_size(c.oid)) total,
       pg_size_pretty(pg_relation_size(c.oid)) heap,
       pg_size_pretty(pg_indexes_size(c.oid)) indexes
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r','p')
ORDER BY pg_total_relation_size(c.oid) DESC
LIMIT 30;
```

附录 C：容量与上线检查清单

C.1 数据模型

- 主键是否短、稳定、不可变。
- 金额、时间、时区、字符集和 NULL 语义是否明确。
- 是否存在无主键表、超宽行、无限增长 JSON。
- 唯一约束、外键、CHECK 与业务幂等键是否落库。

C.2 索引

- 高频查询是否同时考虑过滤、排序和返回列。
- 是否有重复/前缀重复索引。
- 写热点表是否索引过多。
- PG 部分/表达式索引的谓词是否与查询匹配。
- MySQL 主键宽度是否放大所有二级索引。

C.3 事务

- 是否有跨远程调用的事务。
- 是否有长事务、空闲事务、大批量单事务。
- 死锁/序列化失败是否有事务级重试。
- 超时是否从网关到数据库一致规划。

C.4 高可用

- RPO/RTO 是否量化。
- 自动切换是否有 fencing。
- Read-after-write 如何处理。
- 连接池是否能快速淘汰旧主连接。
- 是否演练 failover 和 failback。

C.5 备份

- 全量备份和 binlog/WAL 是否连续。
- 是否异地、加密、不可变。
- 是否恢复过随机历史备份。
- 恢复是否包含用户、权限、扩展和参数。
- 实测恢复时间是否满足 RTO。

附录 D：面试中的常见低质量回答

1. “用了索引就一定快。” 忽略选择性、回表、缓存、排序和写成本。
2. “RR 完全没有幻读。” 没有区分快照读、当前读和业务竞态。
3. “复制等于备份。” 忽略逻辑错误会复制。
4. “连接池越大吞吐越高。” 忽略数据库并行容量。
5. “PG 必须经常 VACUUM FULL。” 混淆空间复用与文件缩小。
6. “MySQL COUNT(*) 是 O(1)。” 把 MyISAM 历史行为套到 InnoDB。
7. “主从延迟就是 Seconds_Behind_Source/replay_lag。” 忽略日志位点和字节差。
8. “在线 DDL 不锁表。” 忽略 metadata lock、阶段锁和资源竞争。
9. “出现死锁说明数据库有 bug。” 忽略死锁是并发事务正常可恢复结果。
10. “SERIALIZABLE 打开就完了。” 忽略回滚、重试和吞吐代价。

官方参考资料

以下资料用于校验本文的版本语义。阅读时应选择与生产环境一致的版本。

MySQL 8.4 Reference Manual

1. MySQL Releases: Innovation and LTS
<https://dev.mysql.com/doc/refman/8.4/en/mysql-releases.html>
2. The InnoDB Storage Engine
<https://dev.mysql.com/doc/refman/8.4/en/innodb-storage-engine.html>
3. InnoDB Multi-Versioning
<https://dev.mysql.com/doc/refman/8.4/en/innodb-multi-versioning.html>
4. InnoDB Transaction Model and Isolation Levels
<https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-model.html>
<https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html>
5. Locking Reads and Locks Set by SQL Statements
<https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html>
<https://dev.mysql.com/doc/refman/8.4/en/innodb-locks-set.html>
6. Deadlocks in InnoDB
<https://dev.mysql.com/doc/refman/8.4/en/innodb-deadlocks.html>
7. InnoDB Redo Log / Undo Logs / Doublewrite Buffer / Buffer Pool
<https://dev.mysql.com/doc/refman/8.4/en/innodb-redo-log.html>
<https://dev.mysql.com/doc/refman/8.4/en/innodb-undo-logs.html>
<https://dev.mysql.com/doc/refman/8.4/en/innodb-doublewrite-buffer.html>
<https://dev.mysql.com/doc/refman/8.4/en/innodb-buffer-pool.html>
8. EXPLAIN and EXPLAIN ANALYZE
<https://dev.mysql.com/doc/refman/8.4/en/explain.html>
9. Optimizer Statistics and Histograms
<https://dev.mysql.com/doc/refman/8.4/en/optimizer-statistics.html>
10. Multiple-Column Indexes, Covering Indexes and Invisible Indexes
<https://dev.mysql.com/doc/refman/8.4/en/multiple-column-indexes.html>
<https://dev.mysql.com/doc/refman/8.4/en/invisible-indexes.html>
11. Replication, GTID, Semisynchronous Replication and Group Replication
<https://dev.mysql.com/doc/refman/8.4/en/replication.html>
<https://dev.mysql.com/doc/refman/8.4/en/replication-gtids.html>
<https://dev.mysql.com/doc/refman/8.4/en/replication-semisync.html>
<https://dev.mysql.com/doc/refman/8.4/en/group-replication.html>
12. Binary Log and Point-in-Time Recovery
<https://dev.mysql.com/doc/refman/8.4/en/binary-log.html>
<https://dev.mysql.com/doc/refman/8.4/en/point-in-time-recovery.html>
13. Performance Schema and sys Schema
<https://dev.mysql.com/doc/refman/8.4/en/performance-schema.html>
<https://dev.mysql.com/doc/refman/8.4/en/sys-schema.html>

PostgreSQL 18 Documentation

14. PostgreSQL 18 Documentation
<https://www.postgresql.org/docs/current/>
15. Concurrency Control, Transaction Isolation and Explicit Locking
<https://www.postgresql.org/docs/current/mvcc.html>
<https://www.postgresql.org/docs/current/transaction-iso.html>
<https://www.postgresql.org/docs/current/explicit-locking.html>

16. Routine Vacuuming and VACUUM
<https://www.postgresql.org/docs/current/routine-vacuuming.html>
<https://www.postgresql.org/docs/current/sql-vacuum.html>
17. Indexes, Multicolumn, Partial, Expression and Index-Only Scans
<https://www.postgresql.org/docs/current/indexes.html>
<https://www.postgresql.org/docs/current/indexes-multicolumn.html>
<https://www.postgresql.org/docs/current/indexes-partial.html>
<https://www.postgresql.org/docs/current/indexes-expressional.html>
<https://www.postgresql.org/docs/current/indexes-index-only-scans.html>
18. EXPLAIN and Planner Statistics
<https://www.postgresql.org/docs/current/using-explain.html>
<https://www.postgresql.org/docs/current/sql-explain.html>
<https://www.postgresql.org/docs/current/planner-stats.html>
19. Table Partitioning
<https://www.postgresql.org/docs/current/ddl-partitioning.html>
20. Write Ahead Log and Reliability
<https://www.postgresql.org/docs/current/wal.html>
<https://www.postgresql.org/docs/current/wal-reliability.html>
21. High Availability, Load Balancing and Replication
<https://www.postgresql.org/docs/current/high-availability.html>
22. Streaming Replication and Hot Standby
<https://www.postgresql.org/docs/current/warm-standby.html>
<https://www.postgresql.org/docs/current/hot-standby.html>
23. Logical Replication and Replication Slots
<https://www.postgresql.org/docs/current/logical-replication.html>
<https://www.postgresql.org/docs/current/logicaldecoding-explanation.html>
24. Continuous Archiving and Point-in-Time Recovery
<https://www.postgresql.org/docs/current/continuous-archiving.html>
25. pg_basebackup, pg_dump and pg_restore
<https://www.postgresql.org/docs/current/app-pgbasebackup.html>
<https://www.postgresql.org/docs/current/app-pgdump.html>
<https://www.postgresql.org/docs/current/app-pgrestore.html>
26. Monitoring Database Activity and Statistics
<https://www.postgresql.org/docs/current/monitoring.html>
<https://www.postgresql.org/docs/current/monitoring-stats.html>

结束语

高级数据库面试真正考察的不是记住多少参数，而是能否把**正确性、并发、访问路径、存储、日志、复制和故障恢复**串成完整链路。

面对任何数据库问题，可以回到四个基本问题：

1. 数据实际存在哪里，索引指向什么？
2. 并发事务能看到哪个版本，持有什么锁？
3. 修改如何通过日志持久化并传播到副本？
4. 故障后如何证明可以恢复，而不是“理论上应该可以”？

能用证据回答这四个问题，才是从会写 SQL 走向高级数据库工程能力的关键。